

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M909.蛇梯棋

bfs, <https://leetcode.cn/problems/snakes-and-ladders/>

思路：

耗时2h。在完成这道题目时，我首先注意到这个棋盘有点特殊，上面的序号是由下到上折返的形式。因此我创建了一个列表，利用列表的index表示序号，用两个for循环在列表对应的index上保存对应的坐标元组，从而完成矩阵的序号与格点的对应。

随后就是常规的bfs，使用队列来遍历访问的棋盘，使用visited保存走过的坐标。在这里，如果读取到的矩阵index上没有蛇/梯，就将index和步数信息添加到队列中，并添加至visited内；否则，如果index上存在蛇/梯的起点，就把终点的index添加入队列内，同时把起点的index添加入visited内，这是由于蛇/梯不能连跳，终点可能是另一个蛇/梯的起点，如果把终点添加入visited，就可能导致终点处的蛇/梯没有被利用，而造成错误的剪枝。耗时主要来源于处理“终点可能是另一个蛇/梯的起点”的情况。

代码：

```
from typing import List, Optional
from collections import deque
```

```

class Solution:

    def snakesAndLadders(self, board: List[List[int]]) -> int:
        n = len(board)

        def board_to_index(n: int):
            board_of_index = [None] * (n ** 2)
            index = 0
            for i in range(n - 1, -1, -1):
                if (n - 1 - i) % 2 == 0:
                    for j in range(n):
                        board_of_index[index] = (i, j)
                        index += 1
                else:
                    for j in range(n - 1, -1, -1):
                        board_of_index[index] = (i, j)
                        index += 1
            return board_of_index

        index_of_board = board_to_index(n)

        def dfs():
            q = deque([[0, 0]])
            visited = {0}
            while q:
                index0 = q.popleft()
                for i in range(1, 7):
                    index1 = index0[0] + i
                    step = index0[1] + 1
                    if index1 == n**2 - 1:
                        return step
                    elif index1 not in visited:
                        node = board[index_of_board[index1][0]]
                        [index_of_board[index1][1]] - 1
                        visited.add(index1)
                        if node == -2:
                            q.append([index1, step])
                        else:
                            if node == n**2 - 1:
                                return step
                            q.append([node, step])

            return -1
        return dfs()

```

代码运行截图 (至少包含有"Accepted")

通过 217 / 217 个通过的测试用例

Happy l3osefod 提交于 2025.11.25 12:01

官方题解

写题解

执行用时分布

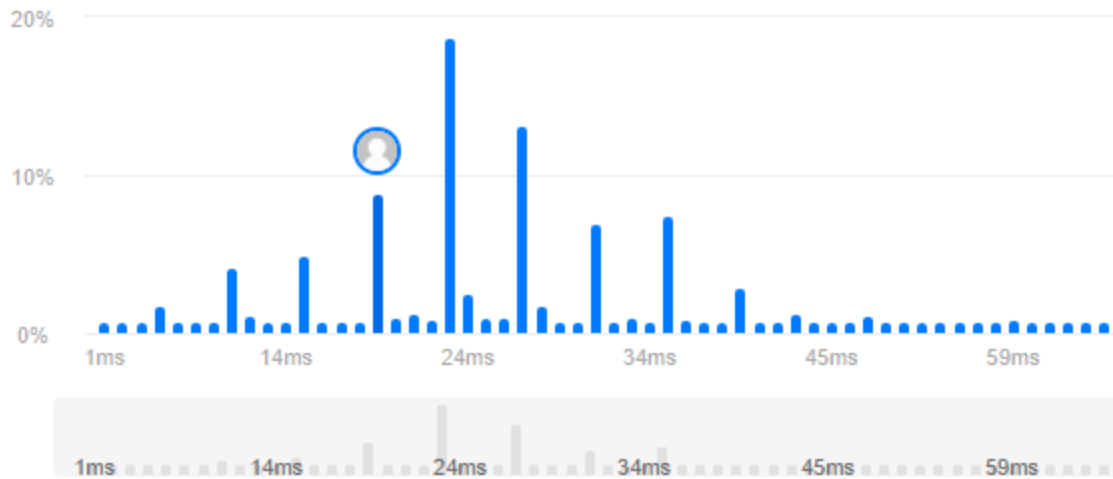
📄

19 ms | 击败 84.45% 🌿

🌟 复杂度分析

消耗内存分布

17.72 MB | 击败 44.16%



代码 | Python3

```
from typing import List, Optional
from collections import deque

class Solution:

    def snakesAndLadders(self, board: List[List[int]]) -> int:
```

sy382: 有向图判环 中等

dfs, topological sort, <https://sunnywhy.com/sfbj/10/3/382>

思路：在有向图判环中，将图的节点和边使用oop的方式存储后，就需要使用dfs写法。这是用于dfs只会深入遍历一条分支，如果在沿有向边进行时发现访问过的节点，就说明图中存在环；而dfs可能存在某个节点同时为有向边的终点的情况，某个点出现在visited内并不能说明有环。耗时0.5h

代码：

```
class Vertex:
    def __init__(self, index=None):
        self.index = index
```

```

        self.out = set()

class Graph:
    def __init__(self):
        self.vertices = dict()

def main():
    n, m = [int(x) for x in input().split()]
    g = Graph()
    for i in range(n):
        g.vertices[i] = Vertex(i)
    for i in range(m):
        vert1, vert2 = [int(x) for x in input().split()]
        g.vertices[vert1].out.add(vert2)

visited = set()
visited_in_one_dfs = []

def dfs(vert: int):
    # print(vert)
    # print(visited, visited_in_one_dfs)
    if vert in visited:
        return False
    elif vert in visited_in_one_dfs:
        return True
    else:
        visited_in_one_dfs.append(vert)
        if len(g.vertices[vert].out) > 0:
            for i in g.vertices[vert].out:
                if dfs(i):
                    return True
        visited.add(visited_in_one_dfs.pop())

for i in g.vertices.keys():
    if dfs(i):
        print('Yes')
        break
else:
    print('No')

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

完美通过

100% 数据通过测试

运行时长: 0 ms

语言: Python

```
1 class Vertex:
2     def __init__(self, index=None):
3         self.index = index
4         self.out = set()
5
6
7 class Graph:
8     def __init__(self):
9         self.vertices = dict()
10
11
12 def main():
13     n, m = [int(x) for x in input().split()]
14     g = Graph()
15     for i in range(n):
16         g.vertices[i] = Vertex(i)
17     for i in range(m):
18         vert1, vert2 = [int(x) for x in input().split()]
19         g.vertices[vert1].out.add(vert2)
20
21
```

M28046: 词梯

bfs, <http://cs101.openjudge.cn/practice/28046/>

思路:

这道题目同样分为两步: 1.把输入的单词转化成图; 2.使用dfs探查两个节点间的最短路径。

第二步比较常规, 在这道题里面比较有收获的是第一步, 就是利用通配符来对只相差一个字母的单词进行判断。具体而言, 创建一个字典patterns对每个单词, 使用for循环依次把单词中的某一位变成“_”作为键, 将单词添加入值中(例如把“bare”添加入“_are”作为键的字典中, 这样所有234位为are的单词就都会被添加入值内), 再依次遍历patterns的键, 对值中的每个单词, 添加其余单词到其邻接表内, 这样就建好了图。耗时1 h

代码:

```
from collections import defaultdict, deque

def build_graph(n: int, words: list, g: dict):
```

```

patterns = defaultdict(set)
for i in range(n):
    temp_word = words[i]
    for j in range(4):
        pattern = temp_word[:j] + '_' + temp_word[j+1:]
        patterns[pattern].add(temp_word)

for pattern in patterns.keys():
    for i in patterns[pattern]:
        g[i] = g[i] | patterns[pattern] - {i}

return g

def bfs(start: str, end: str, graph: dict):
    q = deque([[start]])
    visited = {start}
    while q:
        temp_trace = q.popleft()
        for i in graph[temp_trace[-1]]:
            if i == end:
                temp_trace.append(i)
                return temp_trace
            if i not in visited:
                visited.add(i)
                temp_trace.append(i)
                q.append(temp_trace[:])
                temp_trace.pop()

    return False

def main():
    n = int(input())
    words = []
    g = dict()
    for i in range(n):
        temp_word = input()
        words.append(temp_word)
        g[temp_word] = set()

    g = build_graph(n, words, g)
    start, end = input().split()
    result = bfs(start, end, g)
    if not result:
        print('NO')
    else:
        print(' '.join(result))

```

```
if __name__ == '__main__':  
    main()
```

代码运行截图 (至少包含有"Accepted")

#50995382提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
from collections import defaultdict, deque  
  
def build_graph(n: int, words: list, g: dict):  
    patterns = defaultdict(set)  
    for i in range(n):  
        temp_word = words[i]  
        for j in range(4):  
            pattern = temp_word[:j] + '_' + temp_word[j+1:]  
            patterns[pattern].add(temp_word)  
  
    for pattern in patterns.keys():  
        for i in patterns[pattern]:  
            g[i] = g[i] | patterns[pattern] - {i}  
  
    return g
```

基本信息

#: 50995382
题目: 28046
提交人: 22n2200011816(略约横溪)
内存: 9556kB
时间: 57ms
语言: Python3
提交时间: 2025-11-25 16:50:14

M433.最小基因变化

bfs, <https://leetcode.cn/problems/minimum-genetic-mutation/description/>

思路:

和上一题词梯非常类似，都是使用通配符+邻接表解决。唯一需要注意的是“目标基因”可能不在bank内，需要单独排除这一种情况并输出'-1'。耗时0.5 h。

代码

```
from typing import List, Optional  
from collections import deque, defaultdict  
  
class Solution:  
    def build_graph(self, n: int, words: set, g: dict):  
        patterns = defaultdict(set)  
        for i in words:  
            temp_word = i  
            for j in range(8):  
                pattern = temp_word[:j] + '_' + temp_word[j + 1:]  
                patterns[pattern].add(temp_word)  
  
        for pattern in patterns.keys():  
            for i in patterns[pattern]:  
                g[i] = g[i] | patterns[pattern] - {i}  
  
        return g
```

```

def bfs(self, start: str, end: str, graph: dict):
    q = deque([[start, 0]])
    visited = {start}
    while q:
        temp_node = q.popleft()
        for i in graph[temp_node[0]]:
            if i == end:
                return temp_node[1] + 1
            if i not in visited:
                visited.add(i)
                q.append([i, temp_node[1]+1])
    return -1

def minMutation(self, startGene: str, endGene: str, bank: List[str]) ->
int:
    bank0 = set(bank)
    bank0.add(startGene)
    bank0.add(endGene)
    g = dict()
    for i in bank0:
        g[i] = set()

    g = Solution.build_graph(self, len(bank0), bank0, g)

    if endGene not in bank:
        return -1

    return Solution.bfs(self, startGene, endGene, g)

```

代码运行截图 (至少包含有"Accepted")

通过 20 / 20 个通过的测试用例

Happy l3osefod 提交于 2025.11.25 17:18

官方题解

写题解

执行用时分布

📖

消耗内存分布

0 ms | 击败 100.00% 🏆

17.51 MB | 击败 49.85%

📈 复杂度分析



代码 | Python3

```
from typing import List, Optional
from collections import deque, defaultdict

class Solution:
    def build_graph(self, n: int, words: set, g: dict):
        patterns = defaultdict(set)
        for i in words:
```

M05443: 兔子与樱花

Dijkstra, <http://cs101.openjudge.cn/practice/05443/>

思路:

首先是使用字典建图，利用字典套字典的方式保留两个点之间的距离信息。

随后是学习并手搓了Dijkstra算法，主要通过类dfs的回溯算法解决。具体而言，Dijkstra算法主要思路是：对i和相邻节点j1...jn，记录起点到相邻节点的最优距离和路径，再选取i到相邻节点中距离最近的距离重复以上步骤；如果没有最近的节点，就返回上一个节点访问次近的节点，整体思路很像树的前序遍历，因此使用dfs的方式，对每个节点按照距离大小依次访问相邻节点，并利用visited保存走过的节点防止重复访问即可。

代码

```

from collections import deque

def build_graph(node: set, edge: set):
    graph = dict()
    for i in node:
        graph[i] = dict()
    for i in edge:
        node1, node2, length = i
        length = int(length)
        graph[node1][node2] = length
        graph[node2][node1] = length
    return graph

def dijkstra(graph: dict, nodes: set, start_node, end_node):
    result = dict()
    for i in graph.keys():
        result[i] = [[start_node], float('inf')]
    result[start_node] = [[start_node], 0]

    visited = set()

    def dfs(temp_node):
        if temp_node in visited:
            return False
        visited.add(temp_node)

        related_nodes = []
        for i in graph[temp_node].keys():
            related_node = i
            distance = graph[temp_node][i]
            related_nodes.append([related_node, distance])
            if result[related_node][1] > result[temp_node][1] + distance:
                trace = result[temp_node][0] + [related_node]
                result[related_node] = [trace, result[temp_node][1] +
distance]

        if len(visited) == len(nodes):
            return True

        related_nodes.sort(key=lambda x: x[1])
        for i in related_nodes:
            if i[0] not in visited:
                dfs(i[0])

    dfs(start_node)
    result[start_node] = [[start_node, start_node], 0]

```

```

    return result[end_node]

def main():
    p = int(input())
    node = set()
    for i in range(p):
        node.add(input())

    q = int(input())
    edge = set()
    for i in range(q):
        edge.add(tuple(input().split()))

    graph = build_graph(node, edge)

    r = int(input())
    for i in range(r):
        start_node, end_node = input().split()
        result, _ = dijkstra(graph, node, start_node, end_node)
        if result[0] == result[1]:
            print(result[0])
        else:
            ans = [result[0]]
            for j in range(1, len(result)):
                last_node = result[j-1]
                now_node = result[j]
                distance = graph[last_node][now_node]
                ans.append(f'({distance})')
                ans.append(now_node)
            print('->'.join(ans))

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#51032451提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
from collections import deque

def build_graph(node: set, edge: set):
    graph = dict()
    for i in node:
        graph[i] = dict()
    for i in edge:
        node1, node2, length = i
        length = int(length)
        graph[node1][node2] = length
        graph[node2][node1] = length
    return graph

def dijkstra(graph: dict, nodes: set, start_node, end_node):
    result = dict()
    for i in graph.keys():
        result[i] = [[start_node], float('inf')]
    result[start_node] = [[start_node], 0]

    visited = set()

    def dfs(temp_node):
        if temp_node in visited:
            return False
        visited.add(temp_node)

        related_nodes = []
```

基本信息

#: 51032451
题目: 05443
提交人: 22n2200011816(略约横溪)
内存: 3800kB
时间: 22ms
语言: Python3
提交时间: 2025-11-27 18:08:18

M28050: 骑士周游

dfs, <http://cs101.openjudge.cn/practice/28050/>

思路:

锻炼了一下图的写法, 利用邻接表的方式存储矩阵上每个index连接的其他index。除此之外就是经典的递归dfs写法, 但是注意有一步贪心: 也就是每次都要访问邻接节点中可前往位置数量最少的节点上, 将可以访问更多位置的节点靠后遍历。当然, 测试数据中也没有给出n较大时无解的起点, 否则即使贪心也是跑不出来的。

代码:

```
def build_graph(n: int):
    direct = [[-2, -1], [-2, 1], [2, -1], [2, 1], [1, 2], [-1, 2], [1, -2], [-1, -2]]
    graph = dict()
    for i in range(n):
        for j in range(n):
            temp_end = set()
            for k in range(8):
                index1_x = i + direct[k][0]
                index1_y = j + direct[k][1]
```

```

        if n > index1_x >= 0 and 0 <= index1_y < n:
            temp_end.add((index1_x, index1_y))
        graph[(i, j)] = temp_end
    return graph

def main():
    n = int(input())
    start_index_x, start_index_y = [int(x) for x in input().split()]
    graph = build_graph(n)

    def dfs(index_x, index_y, cout, visited):
        if cout == n**2:
            return visited
        point_to_visit = []
        for i in graph[(index_x, index_y)]:
            if i not in visited:
                point_to_visit.append(i)
        point_to_visit.sort(key=lambda x: len(graph[x] - (graph[x] &
visited)))
        for i in point_to_visit:
            visited.add(i)
            if dfs(i[0], i[1], cout+1, visited):
                return True
            visited.discard(i)
        return False

    result = dfs(start_index_x, start_index_y, cout=1, visited=
{(start_index_x, start_index_y)})
    if result:
        print('success')
    else:
        print('fail')

if __name__ == '__main__':
    main()

```

状态: Accepted

源代码

```
from typing import List, Optional
from collections import deque, defaultdict

def build_graph(n: int):
    direct = [[-2, -1], [-2, 1], [2, -1], [2, 1], [1, 2], [-1, 2], [1, -2], [-1, -2]]
    graph = dict()
    for i in range(n):
        for j in range(n):
            temp_end = set()
            for k in range(8):
                index1_x = i + direct[k][0]
                index1_y = j + direct[k][1]
                if n > index1_x >= 0 and 0 <= index1_y < n:
                    temp_end.add((index1_x, index1_y))
            graph[(i, j)] = temp_end
    return graph

def main():
    n = int(input())
    start_index_x, start_index_y = [int(x) for x in input().split()]
    graph = build_graph(n)
```

基本信息

#:	51030925
题目:	28050
提交人:	22n2200011816(略约横溪)
内存:	4984kB
时间:	31ms
语言:	Python3
提交时间:	2025-11-27 16:49:36

2. 学习总结和个人收获

这周的图题目其实和计概学的dfs/bfs关联很大，相对来说思路好想了很多（虽然由于不少题目有一些情况没考虑，还是耗费了一些时间），总体是比链表和树难度低一些。